MODULES 5 & 6: PROJECT PART 1

GitHub Repository Link: https://github.com/devcnx/csd430

styles.css

```
/*
    styles.css - External CSS for Module 4 assignment
    Author: Brittaney Perry-Morgan
    Description: Styles for the JavaBean movie data display.
 */

/* ==========================
   1. GLOBAL STYLES
   ========================== */

*,
html,
body {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: 'Segoe UI', Arial, sans-serif;
    background: linear-gradient(120deg, #f8fafc 0%, #e0e7ef 100%);
    margin: 0;
    padding: 20px;
    min-height: 100vh;
}

/* ==========================
   2. LAYOUT & CONTAINER
   ========================== */

.container,
.top-title {
    max-width: 900px;
    margin: 0 auto;
    padding: 20px;
}
```

```
.container {
    background: #fff;
    border-radius: 12px;
    box-shadow: 0 4px 24px rgba(0, 0, 0, 0.08);
    margin-bottom: 20px;
}

.top-title {
    text-align: center;
    background: linear-gradient(135deg, #2a3a5e 0%, #1a2540 100%);
    color: #fff;
}

.top-title h1 {
    color: #fff;
    margin: 0 0 10px 0;
}

.top-title h2 {
    color: #e0e7ef;
    margin: 0;
    font-weight: 400;
}

.top-title h2 em {
    font-style: normal;
    color: #a3d9ff;
}

/* ========================
   3. TYPOGRAPHY
   ======================== */

h1,
h2,
h3 {
    color: #2a3a5e;
    margin: 20px 0 15px 0;
    text-transform: uppercase;
}
```

```
h1 {
    font-weight: 200;
    font-size: 1.8rem;
}

h2 {
    font-weight: 400;
    font-size: 1.4rem;
}

h3 {
    font-weight: 600;
    font-size: 1.1rem;
    color: #1a2540;
    border-bottom: 2px solid #e3eaf6;
    padding-bottom: 8px;
}

p.description {
    color: #ffffff;
    line-height: 1.7;
    margin: 15px auto;
    padding: 0 20px;
}

/* ========================
   4. LISTS
   ======================== */

.field-descriptions {
    list-style: none;
    padding: 0 20px;
}

.field-descriptions li {
    padding: 10px 15px;
    margin: 8px 0;
    background: #f1f5fa;
    border-radius: 6px;
    border-left: 4px solid #2a3a5e;
    color: #444;
```

```
}

/* ========================
   5. FORMS
   ========================= */

.form-table {
    width: 100%;
    border-collapse: collapse;
    margin: 20px;
    background: #fff;
}

.form-table tr {
    background: #fff;
}

.form-table td {
    border: 1px solid #e3eaf6;
    padding: 12px 15px;
    vertical-align: top;
}

.form-table td:first-child {
    background: #f9fbfd;
    width: 25%;
    font-weight: 600;
    color: #1a2540;
    vertical-align: middle;
}

.form-table td:last-child {
    width: 75%;
}

.form-table label {
    display: block;
    margin-bottom: 5px;
    font-size: 0.9rem;
    color: #333;
}
```

```
.form-table input[type="text"],
.form-table input[type="email"],
.form-table input[type="tel"],
.form-table input[type="number"],
.form-table input[type="date"],
.form-table select,
.form-table textarea {
    width: 100%;
    padding: 10px 12px;
    border: 1px solid #ccc;
    border-radius: 6px;
    font-size: 1rem;
    font-family: 'Segoe UI', Arial, sans-serif;
    box-sizing: border-box;
}

.form-table select {
    background: #fff;
    cursor: pointer;
}

.form-table textarea {
    resize: vertical;
    min-height: 80px;
}

.form-table input[type="checkbox"] {
    width: auto;
    margin-right: 8px;
}

/* ========================
   6. NAVIGATION
   ======================== */

nav ul {
    list-style: none;
    padding: 0 20px;
}
```

```
nav ul li {
    margin: 10px 0;
}

nav ul li a {
    color: #2a3a5e;
    text-decoration: none;
    font-weight: 600;
    padding: 8px 12px;
    background: #f1f5fa;
    border-radius: 6px;
    transition: all 0.2s ease;
}

nav ul li a:hover {
    background: #2a3a5e;
    color: #fff;
}

.nav-desc {
    display: block;
    color: #666;
    font-size: 0.85rem;
    margin-top: 8px;
    font-weight: 400;
}

/* ========================
   6. FORM ACTIONS (for button links)
   ========================= */

.form-actions {
    margin: 30px 20px;
    padding: 15px;
    background: #f9fbfd;
    border-radius: 8px;
    text-align: center;
}

.btn-submit,
.btn-reset {
```

```
    padding: 12px 24px;
    margin: 0 10px;
    font-size: 1rem;
    font-weight: 600;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    transition: all 0.2s ease;
    text-decoration: none;
    display: inline-block;
}

.btn-submit {
    background: #2a3a5e;
    color: #fff;
}

.btn-submit:hover {
    background: #1a2540;
    transform: translateY(-1px);
    box-shadow: 0 4px 12px rgba(42, 58, 94, 0.3);
}

.btn-reset {
    background: #e3eaf6;
    color: #2a3a5e;
}

.btn-reset:hover {
    background: #cdd8eb;
}

/* ========================
   7. FORM ACTIONS (for button links)
   ========================= */

.form-actions {
    margin: 30px 20px;
    padding: 15px;
    background: #f9fbfd;
    border-radius: 8px;
```

```
    text-align: center;
}

.btn-submit,
.btn-reset {
    padding: 12px 24px;
    margin: 0 10px;
    font-size: 1rem;
    font-weight: 600;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    transition: all 0.2s ease;
    text-decoration: none;
    display: inline-block;
}

.btn-submit {
    background: #2a3a5e;
    color: #fff;
}

.btn-submit:hover {
    background: #1a2540;
    transform: translateY(-1px);
    box-shadow: 0 4px 12px rgba(42, 58, 94, 0.3);
}

.btn-reset {
    background: #e3eaf6;
    color: #2a3a5e;
}

.btn-reset:hover {
    background: #cdd8eb;
}

/* ==========================
   8. TABLES
   ========================== */
```

```
table {
    width: 100%;
    border-collapse: collapse;
    margin: 20px;
    background: #fff;
}

thead {
    background: #2a3a5e;
    color: #fff;
}

th,
td {
    border: 1px solid #e3eaf6;
    padding: 12px 15px;
    text-align: left;
}

th {
    font-weight: 600;
    font-size: 0.85rem;
    text-transform: uppercase;
    letter-spacing: 0.5px;
}

td {
    font-size: 0.95rem;
    color: #333;
    vertical-align: top;
}

tr:nth-child(even) {
    background: #f9fbfd;
}

tr:hover {
    background: #e8f0ff;
    transition: background 0.2s ease;
}
```

```
/* ========================
   9. FOOTER
   ======================== */

footer {
    text-align: center;
    padding: 20px;
    color: #666;
    font-size: 0.9rem;
}

footer p {
    margin: 0;
}

/* ========================
   10. RESPONSIVE DESIGN
   ======================== */

@media (max-width: 768px) {
    body {
        padding: 10px;
    }

    .container,
    .top-title {
        padding: 15px;
    }

    h1 {
        font-size: 1.4rem;
    }

    h2 {
        font-size: 1.1rem;
    }

    h3 {
        font-size: 1rem;
    }
```

```
.form-table {
    margin: 10px 0;
    font-size: 0.85rem;
}

.form-table td {
    padding: 8px 10px;
}

.form-table td:first-child {
    font-size: 0.85rem;
}

.form-table input[type="text"],
.form-table input[type="email"],
.form-table input[type="tel"],
.form-table input[type="number"],
.form-table input[type="date"],
.form-table select,
.form-table textarea {
    font-size: 0.9rem;
    padding: 8px;
}

table {
    margin: 10px 0;
    font-size: 0.85rem;
}

th,
td {
    padding: 8px 10px;
}

p.description {
    padding: 0 10px;
}

.field-descriptions {
    padding: 0 10px;
}
```

```
    .field-descriptions li {
        padding: 8px 12px;
    }

    nav ul {
        padding: 0 10px;
    }

    .form-actions {
        margin: 20px 10px;
        padding: 10px;
    }

    .btn-submit,
    .btn-reset {
        padding: 10px 16px;
        font-size: 0.9rem;
        margin: 5px;
    }
}
```

index.jsp

```
<%--
    CSD430 Server-Side Development

    Module 5 & 6 Project Part 1: CRUD-READ Operations
    Author: Brittaney Perry-Morgan

    Description: Course index page with navigation to movie database
    application demonstrating JDBC connectivity and JavaBeans.

    index.jsp - Course Index Page
--%>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Module 5 & 6 Project Part 1 - CRUD-READ Operations</title>
    <link rel="stylesheet" type="text/css" href="css/styles.css">
```

```
</head>
<body>
  <div class="container top-title">
    <h1>Module 5 & 6 Assignment</h1>
    <h2>JDBC & JavaBeans - <em>CRUD READ Operations</em></h2>
    <p class="description">Author: Brittaney Perry-Morgan</p>
  </div>

  <div class="container">
    <h3>Movie Database CRUD Operations</h3>
    <p class="description">
      This web application demonstrates database connectivity using JDBC
      and JavaBeans for Create, Read, Update, and Delete (CRUD) operations.
    </p>

    <h3>Database Information</h3>
    <ul class="field-descriptions">
      <li><strong>Database Name:</strong> CSD430</li>
      <li><strong>Table Name:</strong> brittaney_movies_data</li>
      <li><strong>Username:</strong> student1</li>
      <li><strong>Topic:</strong> Movies</li>
      <li><strong>Records:</strong> 10 movie records with 6 fields each</li>
    </ul>

    <h3>Navigation - READ Operations</h3>
    <nav>
      <ul>
        <li>
          <a href="movieSelection.jsp">Movie Selection Form</a>
          <span class="nav-desc">Select a movie ID from dropdown to view details</span>
        </li>
      </ul>
    </nav>
  </div>

  <footer>
    <p>&copy; 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development</p>
  </footer>
</body>
</html>
```

MovieBean.java

```java
package csd430;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.IOException;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

/**
 * MovieBean - JDBC JavaBean for CRUD-READ Operations.
 *
 * <p>
 * This JavaBean provides database connectivity using JDBC to retrieve movie
 * records from a MySQL database. It implements the READ operations for a
 * CRUD application, specifically:
 * </p>
 * <ul>
 *   <li>Retrieving all movie IDs for populating a dropdown menu</li>
 *   <li>Retrieving a single movie record by its primary key</li>
 * </ul>
 *
 * <p>
 * The bean follows JavaBean conventions with getter methods for each property
 * and uses PreparedStatement to prevent SQL injection attacks.
 * </p>
 *
 * @author Brittaney Perry-Morgan
 * @version 1.0
 * @since CSD430 Module 5 & 6 Project Part 1
 * @see java.io.Serializable
 */
public class MovieBean {

    // ==========================
    // Database Configuration
```

```
// ========================

/** Database connection URL (e.g., jdbc:mysql://localhost:3306/CSD430) */
private static String DB_URL;

/** Database username for authentication */
private static String DB_USER;

/** Database password for authentication */
private static String DB_PASSWORD;

/**
 * Static initializer block that loads database configuration.
 *
 * <p>
 * Configuration is loaded in the following order of precedence:
 * </p>
 * <ol>
 *   <li>Environment variables (DB_URL, DB_USER, DB_PASSWORD)</li>
 *   <li>.env file in the application root directory</li>
 *   <li>Default fallback values</li>
 * </ol>
 */
static {
    loadEnvironmentVariables();
}

/**
 * Loads database configuration from environment variables or .env file.
 *
 * <p>
 * This method attempts to read configuration from system environment
 * variables first. If not found, it looks for a .env file in the
 * application directory. If neither source provides configuration,
 * default values are used.
 * </p>
 */
private static void loadEnvironmentVariables() {
    // Try environment variables first
    DB_URL = System.getenv("DB_URL");
    DB_USER = System.getenv("DB_USER");
```

```
    DB_PASSWORD = System.getenv("DB_PASSWORD");

    // Fall back to .env file if environment variables not set
    if (DB_URL == null || DB_USER == null || DB_PASSWORD == null) {
        String envFile = System.getProperty("user.dir") + "/.env";
        File file = new File(envFile);

        if (file.exists()) {
            try (BufferedReader reader = new BufferedReader(new FileReader(file))) {
                String line;
                while ((line = reader.readLine()) != null) {
                    // Skip empty lines and comments
                    if (line.trim().isEmpty() || line.trim().startsWith("#")) {
                        continue;
                    }

                    String[] parts = line.split("=", 2);
                    if (parts.length == 2) {
                        String key = parts[0].trim();
                        String value = parts[1].trim();

                        switch (key) {
                            case "DB_URL":
                                DB_URL = value;
                                break;
                            case "DB_USER":
                                DB_USER = value;
                                break;
                            case "DB_PASSWORD":
                                DB_PASSWORD = value;
                                break;
                            default:
                                // Ignore unknown keys
                                break;
                        }
                    }
                }
            } catch (IOException e) {
                System.err.println("Error loading .env file: " + e.getMessage());
            }
        }
```

```
    }

    // Apply default values if still null
    if (DB_URL == null) {
        DB_URL = "jdbc:mysql://localhost:3306/CSD430";
    }
    if (DB_USER == null) {
        DB_USER = "student1";
    }
    if (DB_PASSWORD == null) {
        DB_PASSWORD = "pass";
    }
}

// ==========================
// Instance Fields
// ==========================

/** Active database connection for this bean instance */
private Connection connection;

/** List of movie IDs retrieved from the database */
private List<Integer> movieIds;

/** ResultSet containing the current movie record */
private ResultSet resultSet;

/** Error message from the last failed operation */
private String errorMessage;

// ==========================
// Constructors
// ==========================

/**
 * Constructs a new MovieBean instance.
 *
 * <p>
 * Initializes the movie IDs list and sets the error message to null.
 * Database connection is established lazily when needed.
 * </p>
```

```
 */
public MovieBean() {
    movieIds = new ArrayList<>();
    errorMessage = null;
}

// ========================
// Database Connection Methods
// ========================

/**
 * Establishes a connection to the MySQL database.
 *
 * <p>
 * This method loads the MySQL JDBC driver and creates a connection
 * using the configured URL, username, and password. It supports
 * both the newer com.mysql.cj.jdbc.Driver and legacy com.mysql.jdbc.Driver.
 * </p>
 *
 * @return {@code true} if connection was successful, {@code false} otherwise
 */
public boolean connect() {
    try {
        // Try newer driver first, fall back to legacy driver
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
        } catch (ClassNotFoundException e) {
            Class.forName("com.mysql.jdbc.Driver");
        }

        connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
        return true;

    } catch (ClassNotFoundException e) {
        errorMessage = "MySQL Driver not found: " + e.getMessage();
        return false;
    } catch (SQLException e) {
        errorMessage = "Database connection error: " + e.getMessage();
        return false;
    }
}
```

```java
/**
 * Closes the database connection and releases resources.
 *
 * <p>
 * This method should be called when the bean is no longer needed
 * to prevent resource leaks. It safely handles null connections
 * and result sets.
 * </p>
 */
public void disconnect() {
    try {
        if (resultSet != null) {
            resultSet.close();
            resultSet = null;
        }
        if (connection != null) {
            connection.close();
            connection = null;
        }
    } catch (SQLException e) {
        errorMessage = "Error closing connection: " + e.getMessage();
    }
}

// ========================
// Data Retrieval Methods
// ========================

/**
 * Retrieves all movie IDs from the database.
 *
 * <p>
 * This method queries the database for all movie_id values and
 * populates the internal movieIds list. The IDs are ordered
 * ascending for consistent dropdown display.
 * </p>
 *
 * <p>
 * If no connection exists, this method will attempt to connect
 * automatically.
```

```
 * </p>
 *
 * @return {@code true} if IDs were retrieved successfully,
 *         {@code false} if an error occurred
 */
public boolean getAllMovieIds() {
    // Ensure connection exists
    if (connection == null) {
        if (!connect()) {
            return false;
        }
    }

    try {
        String query = "SELECT movie_id FROM brittaney_movies_data ORDER BY movie_id";
        PreparedStatement stmt = connection.prepareStatement(query);
        resultSet = stmt.executeQuery();

        // Clear existing list and populate with new IDs
        movieIds.clear();
        while (resultSet.next()) {
            movieIds.add(resultSet.getInt("movie_id"));
        }

        // Clean up resources
        resultSet.close();
        stmt.close();
        return true;

    } catch (SQLException e) {
        errorMessage = "Error fetching movie IDs: " + e.getMessage();
        return false;
    }
}

/**
 * Retrieves a single movie record by its primary key.
 *
 * <p>
 * This method queries the database for a specific movie using
 * the movie_id parameter. If found, the ResultSet is stored
```

```
 * internally and can be accessed via the getter methods.
 * </p>
 *
 * <p>
 * Uses PreparedStatement to prevent SQL injection attacks.
 * </p>
 *
 * @param movieId the primary key of the movie to retrieve
 * @return {@code true} if the movie was found, {@code false} otherwise
 */
public boolean getMovieById(int movieId) {
    // Ensure connection exists
    if (connection == null) {
        if (!connect()) {
            return false;
        }
    }

    try {
        String query = "SELECT * FROM brittaney_movies_data WHERE movie_id = ?";
        PreparedStatement stmt = connection.prepareStatement(query);
        stmt.setInt(1, movieId);
        resultSet = stmt.executeQuery();

        if (!resultSet.next()) {
            errorMessage = "Movie not found with ID: " + movieId;
            resultSet.close();
            stmt.close();
            return false;
        }

        return true;

    } catch (SQLException e) {
        errorMessage = "Error fetching movie: " + e.getMessage();
        return false;
    }
}

// ==========================
// Getter Methods (JavaBean Properties)
```

```
// ========================

/**
 * Returns the list of all movie IDs.
 *
 * <p>
 * This list is populated by the {@link #getAllMovieIds()} method.
 * </p>
 *
 * @return a List of Integer movie IDs, or an empty list if not yet populated
 */
public List<Integer> getMovieIds() {
    return movieIds;
}

/**
 * Returns the movie_id of the current record.
 *
 * <p>
 * This method should only be called after a successful call to
 * {@link #getMovieById(int)}.
 * </p>
 *
 * @return the movie ID, or 0 if an error occurs
 */
public int getMovieId() {
    try {
        return resultSet.getInt("movie_id");
    } catch (SQLException e) {
        return 0;
    }
}

/**
 * Returns the title of the current movie record.
 *
 * @return the movie title, or an empty string if an error occurs
 */
public String getTitle() {
    try {
        return resultSet.getString("title");
```

```
    } catch (SQLException e) {
        return "";
    }
}

/**
 * Returns the genre of the current movie record.
 *
 * @return the movie genre, or an empty string if an error occurs
 */
public String getGenre() {
    try {
        return resultSet.getString("genre");
    } catch (SQLException e) {
        return "";
    }
}

/**
 * Returns the release year of the current movie record.
 *
 * @return the release year, or 0 if an error occurs
 */
public int getReleaseYear() {
    try {
        return resultSet.getInt("release_year");
    } catch (SQLException e) {
        return 0;
    }
}

/**
 * Returns the MPAA rating of the current movie record.
 *
 * @return the rating (e.g., "PG-13", "R"), or an empty string if an error occurs
 */
public String getRating() {
    try {
        return resultSet.getString("rating");
    } catch (SQLException e) {
        return "";
```

```
        }
    }

    /**
     * Returns the director of the current movie record.
     *
     * @return the director name, or an empty string if an error occurs
     */
    public String getDirector() {
        try {
            return resultSet.getString("director");
        } catch (SQLException e) {
            return "";
        }
    }

    // ==========================
    // Error Handling
    // ==========================

    /**
     * Returns the error message from the last failed operation.
     *
     * <p>
     * This method should be called after a method returns {@code false}
     * to determine the cause of the failure.
     * </p>
     *
     * @return the error message, or {@code null} if no error occurred
     */
    public String getErrorMessage() {
        return errorMessage;
    }
}
```

movieDisplay.jsp

```
<%--
    CSD430 Server-Side Development

    Module 5 & 6 Project Part 1: CRUD-READ Operations
    Author: Brittaney Perry-Morgan
```

```
   Description: Displays the selected movie's details in an HTML table
   format with field descriptions.

   movieDisplay.jsp - Movie Details Display Page
--%>
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@ page import="csd430.MovieBean" %>
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Movie Details - CSD430</title>
   <link rel="stylesheet" type="text/css" href="css/styles.css">
</head>
<body>
   <div class="container top-title">
      <h1>Module 5 & 6 Project Part 1</h1>
      <h2>Movie Database - <em>READ Operation</em></h2>
      <p class="description">Displaying selected movie details from the database.</p>
   </div>

   <div class="container">
<%
      String movieIdParam = request.getParameter("movieId");

      if (movieIdParam == null || movieIdParam.isEmpty()) {
%>
      <h3>Error</h3>
      <p class="description error">No movie selected. Please go back and select a movie.</p>
<%
      } else {
         int movieId = Integer.parseInt(movieIdParam);

         MovieBean movieBean = new MovieBean();
         boolean connected = movieBean.connect();

         if (!connected) {
%>
      <h3>Database Connection Error</h3>
      <p class="description error"><%= movieBean.getErrorMessage() %></p>
```

```
<%
        } else {
            boolean success = movieBean.getMovieById(movieId);

            if (!success) {
%>
    <h3>Error Retrieving Movie</h3>
    <p class="description error"><%= movieBean.getErrorMessage() %></p>
<%
            } else {
%>
    <h3>Movie Details</h3>
    <p class="description">Record ID: <%= movieId %></p>

    <table>
        <thead>
            <tr>
                <th>Movie ID</th>
                <th>Title</th>
                <th>Genre</th>
                <th>Release Year</th>
                <th>Rating</th>
                <th>Director</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td><%= movieBean.getMovieId() %></td>
                <td><%= movieBean.getTitle() %></td>
                <td><%= movieBean.getGenre() %></td>
                <td><%= movieBean.getReleaseYear() %></td>
                <td><%= movieBean.getRating() %></td>
                <td><%= movieBean.getDirector() %></td>
            </tr>
        </tbody>
    </table>

    <h3>Field Descriptions</h3>
    <ul class="field-descriptions">
        <li><strong>Movie ID:</strong> Unique identifier for each movie (Primary Key)</li>
        <li><strong>Title:</strong> The name of the movie</li>
```

```
        <li><strong>Genre:</strong> The category or type of the movie</li>
        <li><strong>Release Year:</strong> The year the movie was released</li>
        <li><strong>Rating:</strong> MPAA rating (G, PG, PG-13, R, etc.)</li>
        <li><strong>Director:</strong> The director of the movie</li>
      </ul>
<%
            }

            movieBean.disconnect();
        }
      }
%>

      <div class="form-actions">
        <a href="movieSelection.jsp" class="btn-submit">Select Another Movie</a>
        <a href="index.jsp" class="btn-reset">Back to Course Index</a>
      </div>
    </div>

    <footer>
      <p>&copy; 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development</p>
    </footer>
</body>
</html>
```

movieSelection.jsp

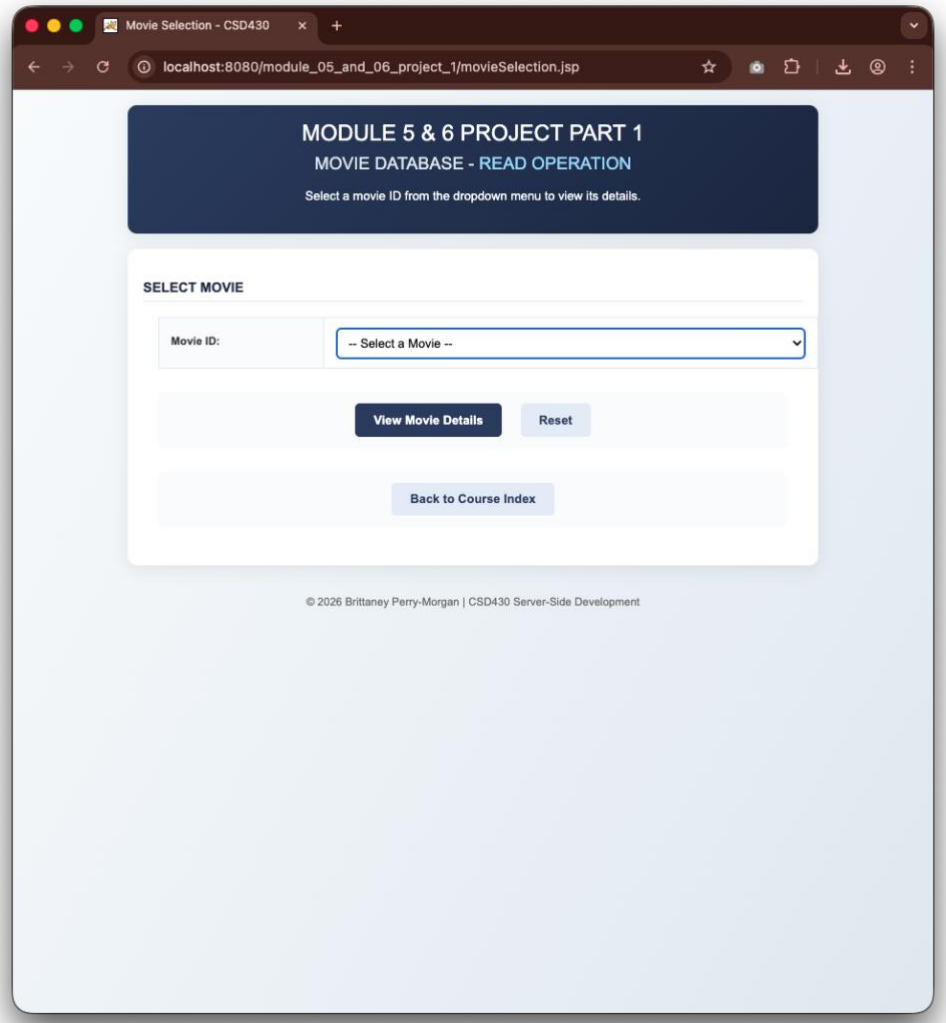
```
<%--
    CSD430 Server-Side Development

    Module 5 & 6 Project Part 1: CRUD-READ Operations
    Author: Brittaney Perry-Morgan

    Description: Displays a dropdown menu with all movie IDs from the
    database. User selects a movie ID and submits to view details.

    movieSelection.jsp - Movie Selection Form Page
--%>
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@ page import="csd430.MovieBean" %>
<%@ page import="java.util.List" %>
<!DOCTYPE html>
<html lang="en">
```

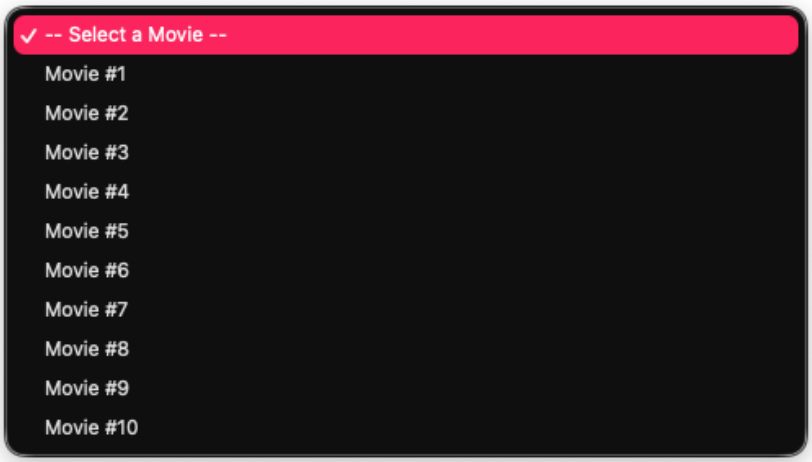
```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Movie Selection - CSD430</title>
    <link rel="stylesheet" type="text/css" href="css/styles.css">
</head>
<body>
    <div class="container top-title">
        <h1>Module 5 & 6 Project Part 1</h1>
        <h2>Movie Database - <em>READ Operation</em></h2>
        <p class="description">Select a movie ID from the dropdown menu to view its details.</p>
    </div>

    <div class="container">
<%
        MovieBean movieBean = new MovieBean();
        boolean connected = movieBean.connect();

        if (!connected) {
%>
        <h3>Database Connection Error</h3>
        <p class="description error"><%= movieBean.getErrorMessage() %></p>
<%
        } else {
            boolean success = movieBean.getAllMovieIds();

            if (!success) {
%>
        <h3>Error Retrieving Data</h3>
        <p class="description error"><%= movieBean.getErrorMessage() %></p>
<%
            } else {
                List<Integer> movieIds = movieBean.getMovieIds();

                if (movieIds.isEmpty()) {
%>
        <h3>No Records Found</h3>
        <p class="description">No movies found in the database.</p>
<%
                } else {
%>
```

```
    <h3>Select Movie</h3>
    <form action="movieDisplay.jsp" method="get">
      <table class="form-table">
        <tr>
          <td><label for="movieId">Movie ID:</label></td>
          <td>
            <select id="movieId" name="movieId" required>
              <option value="">-- Select a Movie --</option>
<%
          for (Integer id : movieIds) {
%>
              <option value="<%= id %>">Movie #<%= id %></option>
<%
          }
%>
            </select>
          </td>
        </tr>
      </table>

      <div class="form-actions">
        <button type="submit" class="btn-submit">View Movie Details</button>
        <button type="reset" class="btn-reset">Reset</button>
      </div>
    </form>
<%
        }
      }

      movieBean.disconnect();
    }
%>

    <div class="form-actions">
      <a href="index.jsp" class="btn-reset">Back to Course Index</a>
    </div>
  </div>

  <footer>
    <p>&copy; 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development</p>
  </footer>
```

```
</body>
</html>
```

deliverables


Movie Selection - CSD430
localhost:8080/module_05_and_06_project_1/movieSelection.jsp
MODULE 5 & 6 PROJECT PART 1
MOVIE DATABASE - READ OPERATION
Select a movie ID from the dropdown menu to view its details.
SELECT MOVIE
Movie ID:
-- Select a Movie --
View Movie Details
Reset
Back to Course Index
© 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development


-- Select a Movie --
Movie #1
Movie #2
Movie #3
Movie #4
Movie #5
Movie #6
Movie #7
Movie #8
Movie #9
Movie #10

MODULE 5 & 6 PROJECT PART 1

MOVIE DATABASE - READ OPERATION

Displaying selected movie details from the database.

MOVIE DETAILS

MOVIE ID	TITLE	GENRE	RELEASE YEAR	RATING	DIRECTOR
2	The Godfather	Crime	1972	R	Francis Ford Coppola

FIELD DESCRIPTIONS

- **Movie ID:** Unique identifier for each movie (Primary Key)
- **Title:** The name of the movie
- **Genre:** The category or type of the movie
- **Release Year:** The year the movie was released
- **Rating:** MPAA rating (G, PG, PG-13, R, etc.)
- **Director:** The director of the movie

Select Another Movie | Back to Course Index

© 2026 Brittaney Perry-Morgan | CSD430 Server-Side Development